

DAY 5

FastAPI - Handling Errors & Body Updates

1. Handling Errors

Client errors use HTTP status codes in the **400 range**. FastAPI's **HTTPException** is the standard way to return them.

HTTPException basics

HTTPException is a Python exception with API-relevant data. You **raise** it rather than return it. Raising it stops execution immediately and sends the error response.

```
@app.get("/items/{item_id}")
async def read_item(item_id: str):
    if item_id not in items:
        raise HTTPException(
            status_code=404,
            detail="Item not found",
        )

    return {"item": items[item_id]}
```

The **detail** parameter can be any JSON-able value, including a string, dict, or list.

Custom headers

HTTPException accepts a **headers** dictionary for advanced cases.

```
raise HTTPException(
    status_code=404,
    detail="Item not found",
    headers={"X-Error": "Custom error"},
)
```

Custom exception handlers

Use **@app.exception_handler(SomeException)** to globally handle a custom exception class and return a specific response whenever it is raised.

Overriding default handlers

- **RequestValidationError** - override FastAPI's invalid-request handler to customize the error format.
- **StarletteHTTPException** - override the base HTTPException handler to also catch errors raised internally by Starlette.
- **RequestValidationError.body** - contains the raw invalid body sent by the client.
- FastAPI's default handlers can be reused from **fastapi.exception_handlers** while adding side effects such as logging.

Note: Avoid returning the raw string of RequestValidationError because it can expose internal file/line information.

2. Body - Updates

PUT - full replace

PUT replaces the entire stored record with the input data. Use **jsonable_encoder** to convert a Pydantic model to JSON-storable data before saving.

```
@app.put("/items/{item_id}", response_model=Item)
async def update_item(item_id: str, item: Item):
    update_item_encoded = jsonable_encoder(item)
    items[item_id] = update_item_encoded
    return update_item_encoded
```

Note: Because PUT replaces the whole object, an omitted field falls back to the model's default and can overwrite a previously stored value.

PATCH - partial update

PATCH lets the client send only the fields that should change while leaving the other stored fields untouched.

The `exclude_unset` technique

- Load the stored data into a Pydantic model: **stored_item_model**.
- Use **item.model_dump(exclude_unset=True)** to keep only fields explicitly sent by the client.
- Use **stored_item_model.model_copy(update=update_data)** to create an updated copy.
- Use **jsonable_encoder(updated_item)** before saving.

```
@app.patch("/items/{item_id}")
async def update_item(item_id: str, item: Item) -> Item:
    stored_item_model = Item(**items[item_id])

    update_data = item.model_dump(
        exclude_unset=True
    )

    updated_item = stored_item_model.model_copy(
        update=update_data
    )

    items[item_id] = jsonable_encoder(updated_item)
    return updated_item
```

Notes

- The same **exclude_unset + model_copy** technique can also be used with PUT.
- To allow fields to be omitted, those fields need defaults, commonly **None**.
- For cleaner API design, use separate models such as **ItemCreate** and **ItemUpdate**.

Source: [fastapi.tiangolo.com/tutorial/handling-errors, /body-updates](https://fastapi.tiangolo.com/tutorial/handling-errors/#body-updates)